

THE DIGITAL CLOCK

Hardware Design & Build Guide

This technical guide documents the design, assembly, and firmware implementation of a digital clock. Utilizing the ATmega328P, the project covers multiple display driving, crystal-controlled timing precision, and custom enclosure fabrication.

SPECIFICATION	VALUE
Controller	ATmega328P-PU
Clock Source	16 MHz Crystal
Display Type	4-Digit 7-Segment
Power Supply	5V DC via USB-B

Contents

1	Theory of Operation	2
2	Step 1: Breadboard Testing with Arduino	3
3	Step 2: Preparing the Bare Chip (Bootloader)	5
4	Step 3: Standalone Hardware Testing	6
5	Step 4: Permanent Assembly (Soldering)	8
6	Step 5: 3D Printing the Snap-Fit Enclosure	10

1 Theory of Operation

A digital clock is fundamentally a highly precise counting machine. To build a reliable standalone clock, every single component in our parts list plays a critical role in keeping the system stable, accurate, and safe.

The Brain: 1x ATmega328P Microcontroller

The ATmega328P chip acts as the central processing unit. Using a hardware timer, it independently counts the microscopic pulses from the crystal oscillator. Every time it reaches exactly 16 million, it knows one exact second has passed, updating the display and internal time variables accordingly.

The Heartbeat: Timing & Oscillation

- **1x 16 MHz Crystal Oscillator:** This is the metronome of the circuit. When a voltage is applied, the quartz crystal physically vibrates exactly 16,000,000 times per second.
- **2x 22 pF Capacitors:** Placed on pins 9 and 10 to Ground, these act as shock absorbers. They stabilize the crystal's electrical resonance, ensuring it starts up reliably and maintains its precise frequency without drifting.

Power & Stability

- **1x Type-B Power Supply:** Provides the steady 5V DC required to run the microcontroller and illuminate the LEDs.
- **2x 100 nF Capacitors:** Placed directly across the power pins (7 & 8, and 20 & 22), these act as tiny local energy reserves. They filter out high-frequency noise and supply instant bursts of current during display multiplexing, preventing the chip from freezing or crashing.

Logic Protection: Hardware Reset

- **1x 10 k Ω Resistor:** Tied from 5V to Pin 1 (RESET), this securely "pulls up" the pin's voltage to a constant HIGH state. If left disconnected (floating), Pin 1 acts like an antenna, picking up random room noise that could spontaneously reset your clock to 12:00.

The Display: Multiplexing & Protection

- **4x 7-Segment Displays:** We don't have enough pins to drive every single LED directly. Instead, we tie common segments together and rapidly cycle power to one digit at a time (Multiplexing/Persistence of Vision).
- **7x 330 Ω Resistors:** Placed in series with the segment pins (A-G). LEDs will pull as much current as available until they destroy themselves. These resistors strictly limit the current, protecting both the ATmega328P and the display from burning out.
- **2x Red LEDs & 2x 330 Ω Resistors:** Used for the blinking center colon. Each LED gets its own current-limiting resistor.

User Interface

- **3x Push Buttons:** Used to set the hours, minutes, and mode.

2 Step 1: Breadboard Testing with Arduino

Wiring the Power Rails

First, we need to establish a common power and ground line on the breadboard.

- **Arduino 5V** → Breadboard Positive (+) Rail.
- **Arduino GND** → Breadboard Negative (-) Rail.

Wiring the Display Segments (The "Shapes")

The segments (A through G) are the individual light bars that make up the numbers.

- **Connection:** Connect Arduino Digital Pins **D2 through D8** to the corresponding segment pins (A through G) on the display.
- **The Protection:** You **must** place a **330 Ω resistor** in series with each of these 7 wires. This limits the current flowing from the Arduino, ensuring the LEDs inside the display do not burn out.

Wiring the Digit Selects (The "Multiplexing")

These pins act as the common ground (or power) for each of the four numbers, allowing us to turn them on and off rapidly.

- **Connection:** Connect Arduino Digital Pins **D9, D10, D11, and D12** directly to the Digit 1, Digit 2, Digit 3, and Digit 4 pins on the display, respectively.

Wiring the Colon LEDs (The "Heartbeat")

To separate the hours and minutes, we use two standard red LEDs placed in the center of the display.

- **Connection:** Connect Arduino Analog Pin **A3** to the long legs (Anodes) of both red LEDs.
- **The Protection:** Connect the short legs (Cathodes) of both LEDs to the Breadboard Negative (-) Rail, placing a **330 Ω resistor** in series for each LED to protect them.

Wiring the User Interface (Push Buttons)

We need three buttons to set the clock: Hour, Minute, and Mode. We are using a clever code trick called `INPUT_PULLUP`, which turns on internal resistors inside the Arduino. Because of this, we only need to wire the buttons to Ground, saving us from using external resistors on the breadboard.

- **Hour Button:** Connect one leg to Arduino Analog Pin **A0**. Connect the other leg to the Negative (-) Rail.
- **Minute Button:** Connect one leg to Arduino Analog Pin **A1**. Connect the other leg to the Negative (-) Rail.
- **Mode Button:** Connect one leg to Arduino Analog Pin **A2**. Connect the other leg to the Negative (-) Rail.

With the wiring complete, connect the Arduino to your computer via USB and upload the clock code.

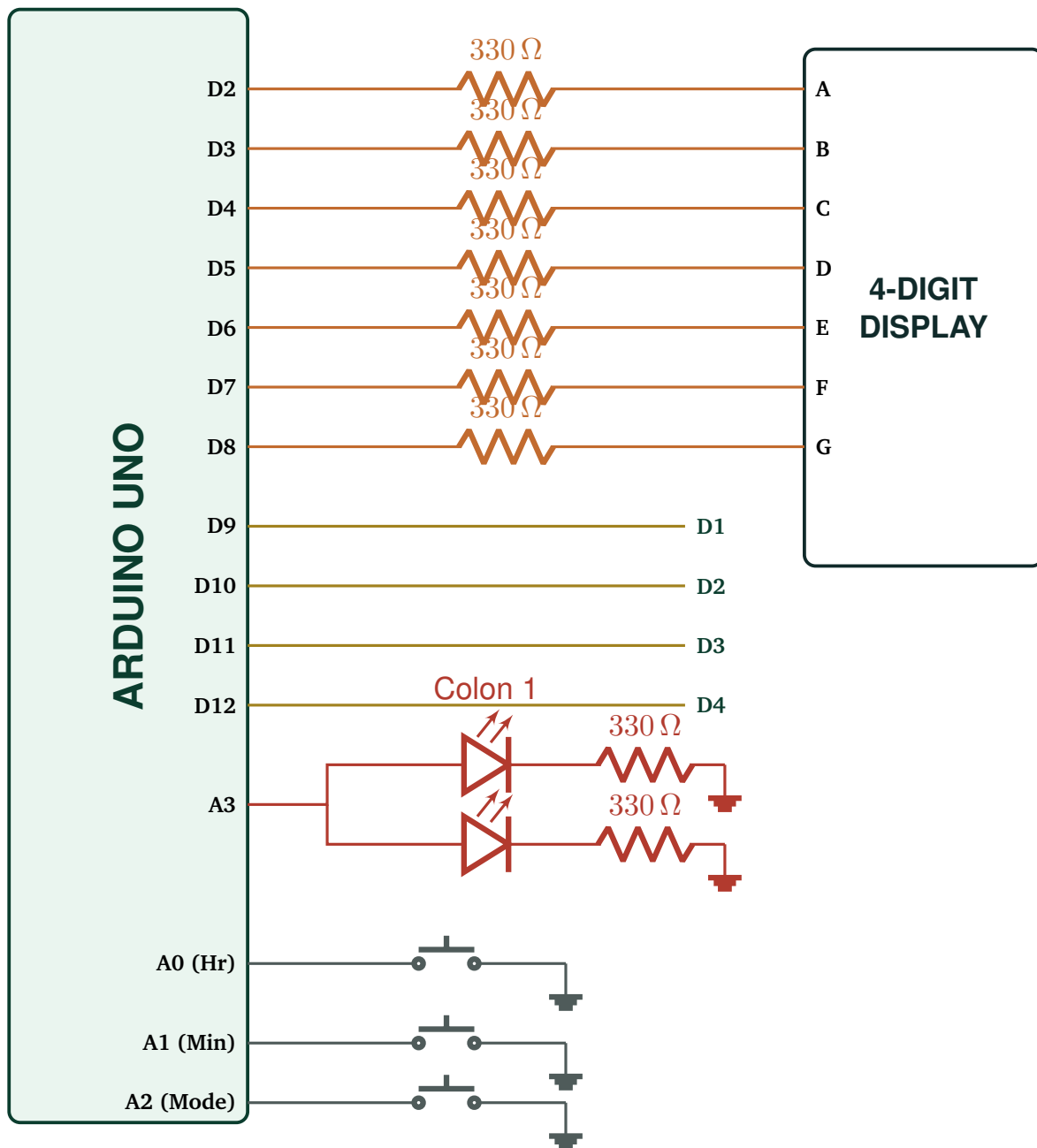


Figure 1: Detailed Electronic Schematic for Prototype Testing.

3 Step 2: Preparing the Bare Chip (Bootloader)

Phase 1: Configure the Arduino Uno as a Programmer

Before we wire the bare chip, we must give the Arduino Uno the instructions it needs to act as a programmer.

1. Plug the standard Arduino Uno into your computer via USB.
2. Open the Arduino IDE. Go to **File > Examples > 11.ArduinoISP > ArduinoISP**.
3. Select your standard Uno board and port, then click **Upload**.
4. Once the upload finishes, **unplug the Arduino from your computer**.

Phase 2: Build the Target Circuit

Place your bare ATmega328P chip straddling the center trench of a blank breadboard.

1. Power & Stability:

- **Power Lines:** Connect Uno **5V** to chip Pins **7 and 20**. Connect Uno **GND** to chip Pins **8 and 22**.
- **Clock Signal:** Place the **16 MHz Crystal** across chip Pins **9 and 10**. Connect a **22 pF capacitor** from Pin 9 to GND, and another from Pin 10 to GND.
- **Target Reset:** Connect a **10 kΩ resistor** from 5V to chip Pin **1 (RESET)**.

2. The Data Lines (ISP):

- **Programmer Reset:** Connect Uno Pin **D10** to chip Pin **1 (RESET)**.
- **MOSI (Master Out Slave In):** Connect Uno Pin **D11** to chip Pin **17**.
- **MISO (Master In Slave Out):** Connect Uno Pin **D12** to chip Pin **18**.
- **SCK (Serial Clock):** Connect Uno Pin **D13** to chip Pin **19**.

To stop automatic reset, place a **10 μF capacitor** on the Arduino Uno itself, connecting its **RESET** pin to its **GND** pin.

Phase 3: Burn the Bootloader

With everything wired, plug the Uno back into your computer.

1. In the Arduino IDE, go to **Tools > Board** and select **Arduino Uno** (This tells the IDE what type of bootloader to put on the bare chip).
2. Go to **Tools > Programmer** and change it from the default to "**Arduino as ISP**".
3. Click **Tools > Burn Bootloader**.

The TX/RX lights on your Uno will flash wildly for about 30 seconds. When you see "Done burning bootloader" at the bottom of the screen, your bare chip is officially ready to become a standalone clock!

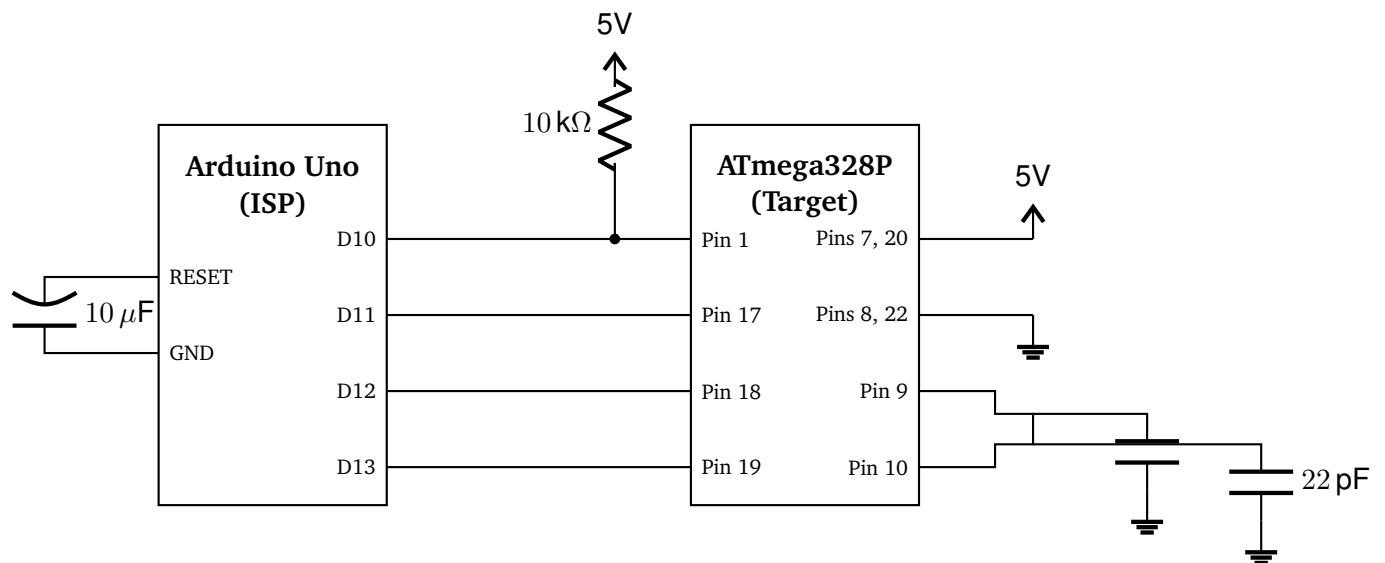


Figure 2: In-System Programming (ISP) Wiring Diagram.

4 Step 3: Standalone Hardware Testing

Now that the ATmega328P has the bootloader, it is ready to run independently. Disconnect the Arduino Uno completely. We will now build the bare chip's "life support" and interface circuitry directly onto the breadboard.

Phase 1: Life Support & Power

Before connecting the display, we must ensure the microcontroller has stable power and precise timing.

1. **Type-B Power Supply:** Connect the 5V output to your breadboard's positive rail, and GND to the negative rail.
2. **Chip Power:** Connect ATmega Pins 7 (VCC) and 20 (AVCC) to the 5V rail. Connect Pins 8 (GND) and 22 (GND) to the GND rail.
3. **Bypass Capacitors:** Place one **100 nF capacitor** directly across Pins 7 and 8. Place the second **100 nF capacitor** across Pins 20 and 22.
4. **Reset Pull-up:** Connect the **10 kΩ resistor** from the 5V rail to Pin 1 (RESET).
5. **The Oscillator:** Place the **16 MHz Crystal** across Pins 9 and 10. Connect one **22 pF capacitor** from Pin 9 to GND, and the other from Pin 10 to GND.

Phase 2: Display & Interface Wiring

Now we attach the peripherals using the same logical layout as our prototype test, but wired directly to the ATmega328P's physical pins.

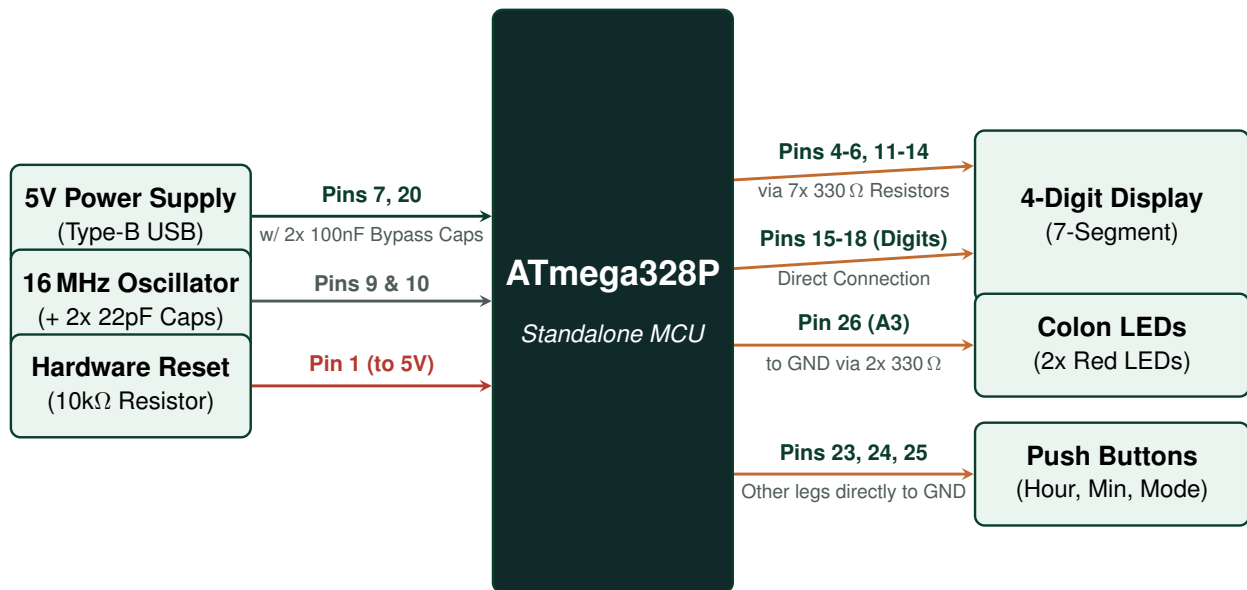


Figure 3: System Block Wiring Diagram for the Standalone Circuit.

1. **7-Segment Display:** Connect ATmega Pins 4, 5, 6, 11, 12, 13, and 14 (which correspond to Arduino D2-D8) to the segment pins A-G on the display. **Crucial:** Place a **330 Ω resistor** in series with each of these 7 lines.
2. **Digit Selects:** Connect ATmega Pins 15, 16, 17, and 18 (Arduino D9-D12) directly to the Digit 1-4 pins on the display.
3. **Colon LEDs:** Connect ATmega Pin 26 (Arduino A3) to the Anodes (long legs) of both **Red LEDs**. Connect the Cathodes (short legs) to GND, placing a **330 Ω resistor** in series for each LED.
4. **Buttons:** Connect the Hour, Minute, and Mode buttons to ATmega Pins 23, 24, and 25 (Arduino A0-A2). Connect the other side of all three buttons directly to GND.

Phase 3: Uploading the Clock Code

To upload the final code without a USB port on the chip, reconnect the SPI programming wires from Step 2 (D10, D11, D12, D13 to the chip). In the Arduino IDE, open your clock code, and instead of clicking the standard upload arrow, click **Sketch > Upload Using Programmer**. Once done, remove the SPI wires. Your standalone clock is now running!

5 Step 4: Permanent Assembly (Soldering)

Once your breadboard prototype is running flawlessly, it is time to make the circuit permanent. We will transfer the components from the breadboard to a perforated circuit board (perfboard) and solder them into place.

Phase 1: Component Placement

Before turning on your soldering iron, dry-fit all your components onto the perfboard to plan your layout.

- Place the 28-pin DIP socket in the center.
- Group the "Life Support" components (Crystal, 22 pF capacitors, 100 nF capacitors, and 10 k Ω resistor) as tightly around the socket as possible. **Note:** The crystal and its capacitors must be placed immediately next to pins 9 and 10 to prevent timing issues.
- Place the display, colon LEDs, and buttons where they will be easily accessible when mounted in the final case.

Phase 2: Soldering the Circuit

Once you are happy with the layout, flip the board over and begin soldering.

1. **Tack it down:** Solder two opposite corners of the DIP socket and the display first so they don't fall out while you work on the rest of the board.
2. **Solder the components:** Solder the legs of all the remaining components. Snip off the excess wire from the resistors and capacitors using flush cutters.
3. **Route the traces:** Using the snipped-off legs from your resistors (or thin, solid-core wire), create the connections between the pins on the underside of the board. Follow the exact same routing you used on the breadboard in Step 3.
4. **The Resistors:** Do not forget to solder the 330 Ω current-limiting resistors inline between the ATmega pins and the display segments!

Phase 3: Final Testing

Visually inspect the underside of your board for "solder bridges"—accidental blobs of solder connecting two adjacent pads that shouldn't be touching. Once verified, insert the ATmega328P into its socket (ensure the notch on the chip matches the notch on the socket). Plug in your 5V power supply, and your permanent clock should light up immediately!



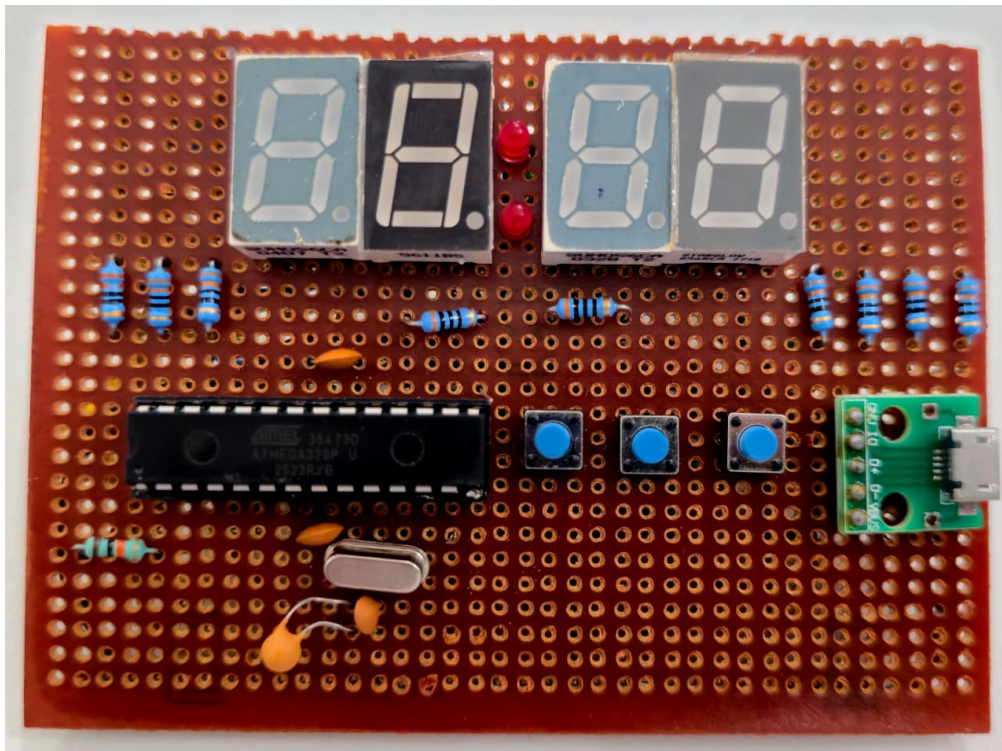


Figure 4: Recommended component layout on the dotboard.

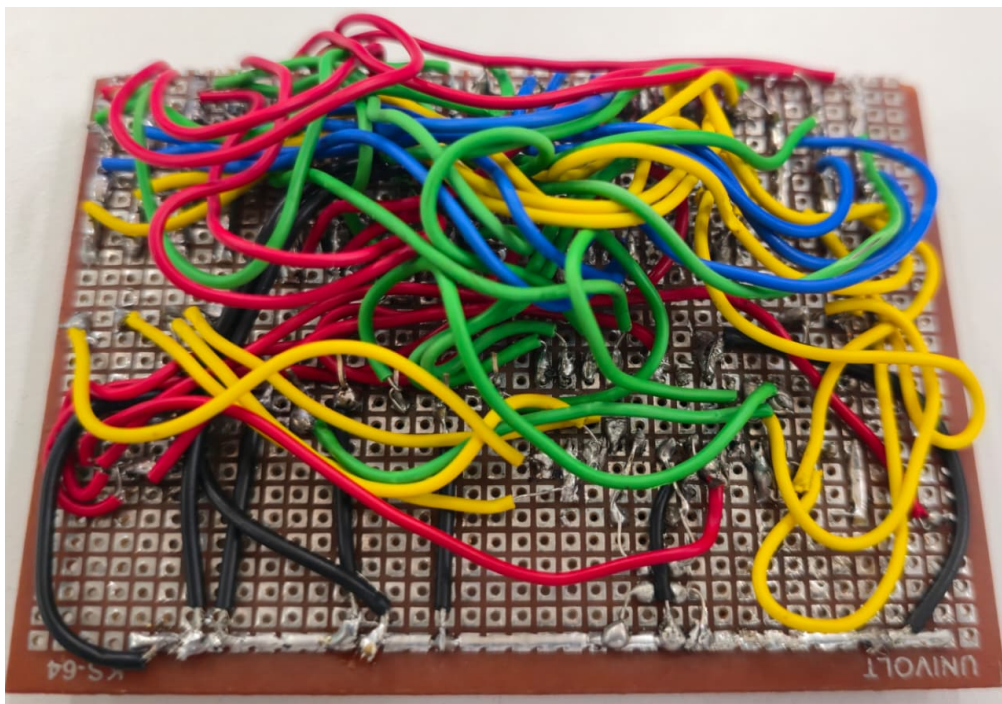


Figure 5: The fully soldered standalone clock back.

6 Step 5: 3D Printing the Snap-Fit Enclosure

To protect our soldered circuit and give the clock a premium, finished look, we are using a custom 3D-printed enclosure. Instead of relying on metal screws or messy glue, this case uses a clever mechanical "snap-fit" design.

The Snap-Fit Mechanism

The enclosure is divided into two parts: a deep bottom tray to house the electronics, and a top lid to frame the display.

- **The Interlock:** Notice the raised sections along the inner walls of the case. These feature tiny horizontal lips. When the two halves are pushed together, the plastic slightly flexes, allowing the lips to slide past each other before snapping back into place, locking the case shut.
- **No Hardware Required:** This design relies entirely on the natural flexibility of thermoplastics like PLA or PETG, making assembly instant and tool-free.

Slicing and Printing Tips

When printing snap-fit joints, precision and layer adhesion are critical. A joint that is too tight won't close, and one that is too loose will fall apart.

1. **Material Choice:** PLA works great for indoor clocks and is stiff enough for a solid "snap." If you want the tabs to be more durable and flexible, consider using PETG.
2. **Layer Height:** Use a finer layer height (like 0.16 mm or 0.20 mm). Snap-fit lips are small details; a coarse layer height might make the locking tabs too rough to slide smoothly.
3. **Orientation:** Print both shells flat on their backs. This ensures the interlocking tabs are built horizontally, which prevents them from snapping off under pressure due to weak layer adhesion.

Final Assembly

Once your parts are printed and cooled off the build plate, it is time for the final assembly:

1. **Insert the Board:** Gently lower your soldered perfboard into the bottom tray. Ensure the power cable has a clear path out of the back of the case and the push buttons align with their designated spots.
2. **Align the Lid:** Place the top lid over the tray, ensuring the cutout perfectly frames the 7-segment display.
3. **The Snap:** Press down firmly and evenly on all four corners of the lid. You should hear a satisfying plastic "click" as the interlocking tabs engage.

Plug in the USB power supply and set the time.

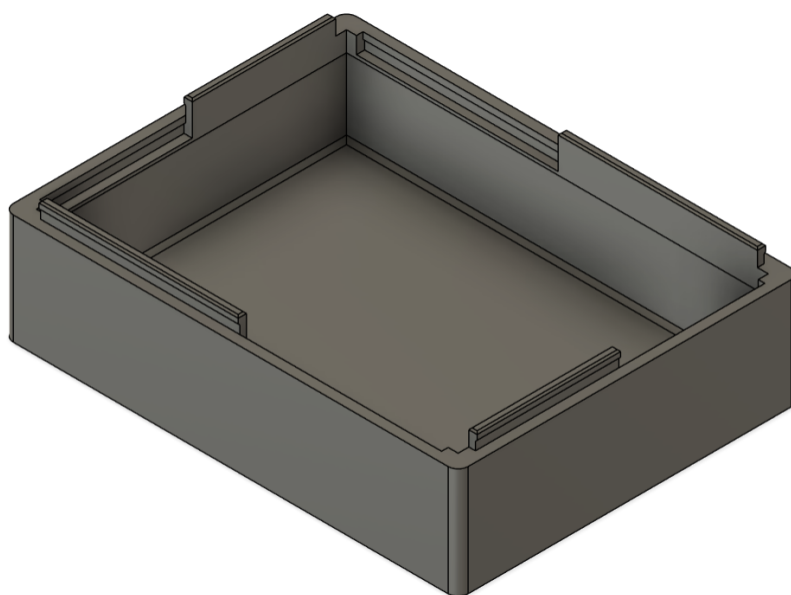


Figure 6: The deep bottom tray.

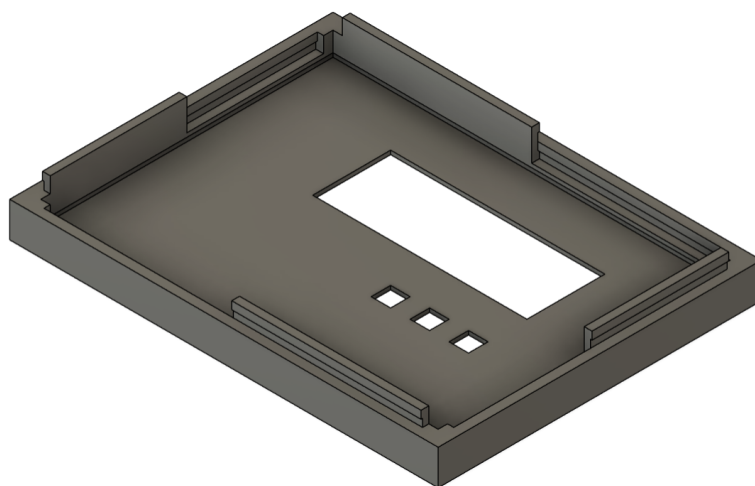


Figure 7: The top lid.